

Moving Visual-Inertial Ordometry into Cross-platform Web for Markerless Augmented Reality

Yakun Huang*, Zhijie Tan*, Xiuquan Qiao†, Jie Zhao, and Fenghua Tian

State Key Laboratory of Networking and Switching Technology, Beijing Univ. of Posts and Telecomm., Beijing, China.

ABSTRACT

Widespread mobile AR implementation still suffers from the unique adaption for different mobile platforms. Enabling AR experience on the cross-platform web is a potential alternative and provides unified implementation. We reveal a lightweight VIO implementation on the web without any visual marker or cumbersome frameworks for AR services. The contributions include 1) designing a novel VIO architecture to balance the user experience and the efficiency; 2) optimizing a single-thread VIO algorithm to avoid the sophisticated multi-threading management and enhancing compatibility; 3) improving the dirty noise of IMU and developing efficient numerical libraries like SuiteSparse for the massive computation afford.

Index Terms: Computing methodologies—Computer graphics—Graphics systems and interfaces—Mixed/augmented reality.

1 INTRODUCTION

Augmented reality (AR) merges a live view of the physical, real-world with a context-sensitive, computer-generated image to provide mixed reality service. However, users have to spend some effort to experience immersive services brought by AR using dedicated devices and specific applications. It is also difficult to quickly share and spread immersive experiences to more users. Mobile web, with its natural cross-platform, no-installation, and shared features, provides us with a powerful tool that implements lightweight AR on the web (i.e., WebAR) with convenient and flexible services [1]. Due to the limited computing capability and the lack of unified standards and specifications, typical WebAR implementation can be divided into two approaches. The first method is based on prior knowledge about the real world. This is the most popular WebAR type, which can be easily implemented with a pure front-end solution because the prior knowledge (usually a visual marker) can solve the tricky problem of world analysis and provide an AR anchor to render. Some well-known third-party web libraries include tracking.js, JSARToolKit, and AR.js for recognizing and tracking, as well as Three.js, Babylon.js for rendering. However, most implementations require designing specific markers, limiting them to more capabilities such as pose estimation, plane detection, and tracking. The second method is exploring SLAM functions on mobile web browsers. WebPoseEstimator [2] optimizes the loosely coupled multi-sensor fusion framework to provide pose estimation service on the web browser. Also, we noticed that the 8th Wall company claimed that they explored using standard JavaScript and WebGL to build, add synchronous positioning, and SLAM functions on mobile web browsers [3]. However, they did not describe the implementation principle in detail due to commercial reasons, which has little promotion for the development of the WebAR research community.

This work focuses on moving visual-inertial odometry (VIO) into the mobile web for markerless AR. First, we propose a lightweight WebVIO framework that can support existing underlying web APIs

and limited computing power, laying the foundation for markerless AR services based on mobile web and embedded WebView apps. WebVIO optimizes a single-thread VIO module and obtains SLAM capability on web browsers by balancing different threads' communication and computational overhead. Second, we evaluate the operability for users and consider the difference between heterogeneous devices and browsers. To get long-term accuracy of pose estimation, we re-design the initialization and the process of a single-thread VIO algorithm to get long-term accuracy of pose estimation. Third, we propose some data augmentation techniques to address the shortcomings of WebSensor in capturing raw data. For implementation, we use advanced technologies such as Emscripten, an LLVM-based source-to-source compiler that can convert C++ code into JavaScript and efficient wasm.

2 DESIGN OF WEBVIO

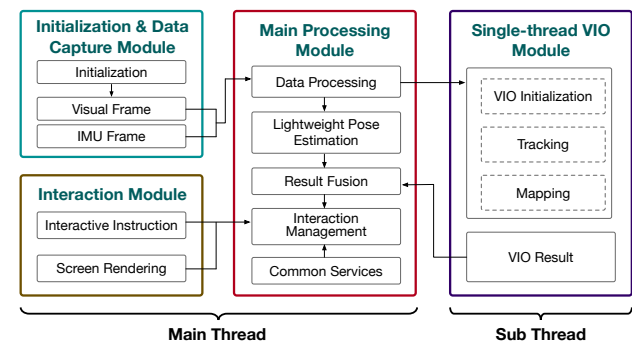


Figure 1: Overview of WebVIO architecture.

We introduce the core components of the WebVIO as shown in Figure 1, including (1) *the initialization and data capture module*. It serves as the entry to WebVIO and is responsible for initializing the VIO parameters and getting the data stream from the sensors. Besides, the module captures live camera frames through the WebRTC and uses WebSensor to capture the IMU sensor, including angle, linear acceleration as raw input samples; (2) *the main processing module*. This module involves the main JavaScript thread (i.e., UI thread) and sub-thread for lightweight VIO. It is fundamentally designed to improve the robustness faced with poor environments; (3) *the single-thread VIO module*. The core VIO module is independent of the main thread by using a web worker to design a single-threaded VIO algorithm that implements Tracking, Mapping, not blocking the main thread. Instead of directly adapting the traditional VIO algorithm involving complex multi-threading to the web, we optimize a single-threaded VIO algorithm for the resource-constrained web platform; (4) *the interaction module*. To interact with users and render the visual models on the web browser, this module is mainly done in the main thread. We present the pipeline of the optimized single-thread VIO for computationally inefficient web in Figure 2, including three fundamental modules of initialization, tracking, and mapping. Compared to VINS-Mono [4] run on the native apps, one of the advantages of WebVIO is single-threaded and low-overhead. Thus, it allows running on general web browsers and embedded browsers. More importantly, WebVIO's initialization module performs much faster on web browsers than VINS-Mono.

*Both authors contributed equally to this research.

†Xiuquan Qiao is the corresponding author. E-mail: qiaoxq@bupt.edu.cn.

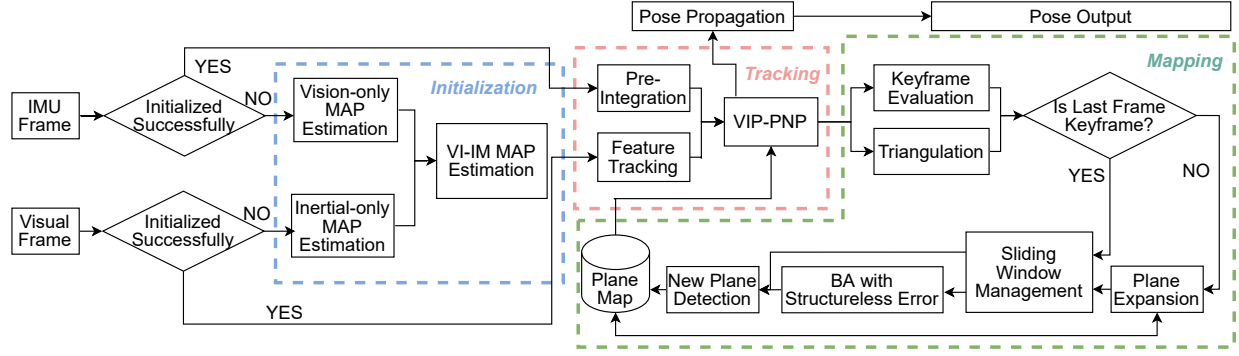


Figure 2: Pipeline of optimized single-thread VIO algorithm.

The data acquired by the IMU sensor may have dirty data, significantly affecting the accuracy of WebVIO. First, we use a complementary filtering method to balance the accelerometer and gyroscope in a long time as follows:

$$angle = \alpha * (pre_angle + gyr * dt) + (1 - \alpha) * (acc), \quad (1)$$

where pre_angle means the calculated previous angle and dt is the duration between two gyroscope readings. acc is the reading of the accelerometer. Both gyr and acc are 3×1 scalars. Note that the parameter α can be fine-tuned to get a better match in practice, and the overhead required for adjustment is also acceptable.

Second, to alleviate the impact of this problem on the VIO algorithm, we use cubic Bezier curves to formulate and sample data that meets the requirements of the VIO frequency as follows:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3, \quad (2)$$

where $t \in [0, 1]$, and we choose four imu data (P_0, P_1, P_2, P_3) and formulate a Bezier curve and then we will sample from this curve to get more data.

3 IMPLEMENTATION AND EXPERIMENTS

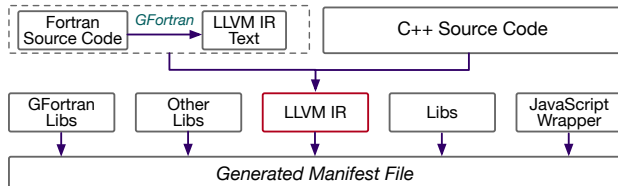


Figure 3: Implementation of compiling Fortran code to wasm code.

We show the overall pipeline of the proposed toolchain for WebVIO in Figure 3. Note that used Flang is a compiler officially supported by the LLVM community. We selected GFortran, a GNU Fortran project, to compile Fortran code into GCC libraries. Then, we compile all of LLVM IR and other standard libraries into wasm code and JS wrapper with the help of such a tool.

To illustrate the correctness, we compare the accuracy of WebVIO in C++ by comparing the RMSE of the absolute localization error with ORB-SLAM2 and PVIO [5] (a VIO method using plane prior) in Table 1. We use the same configurations in, including fine-tuned IMU noise parameters. The results show that: (1) WebVIO outperforms on the EuRoC dataset than on the TUM-VI. WebVIO is more suitable for AR services in a small area, and it is challenging to converge outdoor on the TUM-VI dataset. (2) On the EuRoC data, WebVIO obtains similar accuracy performance as other methods. In particular, WebVIO outperforms PVIO, which performs the initialization overhead of 7s and 15s, respectively. Note that ORB-SLAM2 is too bulky to apply to the mobile web. Thus we do not compare it with WebVIO.

Table 1: The RMSE(m) of different VIO algorithms. "+/-loop" means using loop-closure or not. "+/- plane" means using plane prior or not. Data in parenthesis means this trajectory is less than 80% complete. $\times$ means this trajectory is less than 50% complete.

Datasets		ORB-SLAM2		PVIO		WebVIO	
		-LOOP	+LOOP	-PLANE	+PLANE	-PLANE	+PLANE
EuRoC	MH-01	0.02	0.03	0.19	0.13	0.22	0.15
	MH-02	0.03	0.03	0.16	0.21	0.18	0.22
	MH-03	0.17	0.05	0.31	0.16	0.29	0.14
	V1-01	0.03	0.03	0.10	0.08	0.13	0.10
	V1-02	0.15	0.04	$\times$	0.09	$\times$	0.11
	V1-03	(0.49)	0.10	$\times$	0.16	$\times$	0.17
	V2-01	0.15	0.03	0.11	0.05	0.09	0.08
	V2-02	(0.50)	0.03	$\times$	0.20	$\times$	0.20
TUM-VI	V2-03	0.03	(0.40)	$\times$	0.30	$\times$	0.30
	Room1	$\times$	0.10	1.65	0.26	2.34	0.28
	Room2	$\times$	0.12	0.12	0.15	0.11	0.29
	Room3	$\times$	(0.14)	0.19	0.18	0.20	0.44
	Corridor1	$\times$	$\times$	$\times$	0.23	$\times$	0.33
	Outdoor1	$\times$	$\times$	$\times$	22.26	$\times$	$\times$

4 CONCLUSION

We successfully compile and optimize a single-thread VIO algorithm into wasm code to boost the long-term accuracy of pose estimation, which is a core function of the AR system. This will benefit other engineers to spend more time on algorithm design and use our method to transplant their algorithms on the web. Third, we introduce some different methods on dealing with noisy data and how to compile numeric computation in Fortran code to wasm code to provide an immersive experience.

ACKNOWLEDGMENTS

This research was supported in part by the Funds for International Cooperation and Exchange of NSFC under Grant 61720106007.

REFERENCES

- [1] Xiuquan Qiao, Pei Ren, Schahram Dustdar, Ling Liu, Huadong Ma, and Junliang Chen. Web AR: A promising future for mobile augmented reality—State of the art, challenges, and insights. *Proceedings of the IEEE*, 107(4):651–666, 2019.
- [2] Yakun Huang, Xiuquan Qiao, Zhijie Tan, Jianwei Zhang, and Jiulin Li. WebPoseEstimator: A Fundamental and Flexible Pose Estimation Framework for Mobile Web AR. In *2021 IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW)*, pages 478–479. IEEE, 2021.
- [3] 8th wall. Retrieved Nov 2, 2020 from <https://8thwall.com>.
- [4] Tong Qin, Peiliang Li, and Shaojie Shen. Vins-mono: A robust and versatile monocular visual-inertial state estimator.
- [5] Jinyu Li, Bangbang Yang, Kai Huang, Guofeng Zhang, and Hujun Bao. Robust and efficient visual-inertial odometry with multi-plane priors. In *Pattern Recognition and Computer Vision - Second Chinese Conference, (PRCV)*, pages 283–295. Springer, 2019.